

Comparison of R/Python for XGBoost

1. The following is the result of my execution in Python and R with the stated dataset.

Method used	Dataset size	Testing-set predictive performance (accuracy)	Time taken for the model to be fit (seconds)
XGBoost in Python via scikit-learn and 5-fold CV	100	85.00	0.26
	1000	93.12	0.31
	10000	94.84	0.45
	100000	95.91	1.81
	1000000	96.12	17.27
	10000000	96.10	183.26
XGBoost in R – direct use of xgboost() with simple cross-validation	100	99.80	0.32
	1000	99.90	0.40
	10000	99.92	0.34
	100000	99.93	0.74
	1000000	99.93	4.00
	10000000	99.93	62.50
XGBoost in R – via caret, with 5-fold CV simple cross-validation	100	99.85	8.44
	1000	99.95	11.86
	10000	99.99	30.06
	100000	99.99	217.52
	1000000	NA	NA
	10000000	NA	NA

Please note NA, I could not perform these due to reaching my memory limit of my computer, R Session was crashed multiple times. While, I could have explored data bricks for that, I could not explore that within due time.

2. Considering both accuracy and execution time, I believe using a dataset of size 10,000 or 100,000 provides sufficiently good results, without the need for more computationally expensive runs. From the comparison, it is clear that the accuracy on the test set is better in R compared to Python, while also requiring significantly less time. For instance, Python took 183.26 seconds for the largest dataset, whereas R completed the same in just 62.50 seconds. However, despite the faster execution time in R, the experience of running the code felt slower and more cumbersome compared to Python. In my view, Python offers a more user-friendly environment with better tooling and packages that make the process faster and more efficient overall. That said, R certainly has its own strengths, particularly in statistical modeling and data analysis.